# Predicting the Next State Is Not Enough: JEPA Representations for Lean Theorem Proving

**Aarnav Choudhary**
University of California, Los Angeles
aarnav11@g.ucla.edu

## Abstract

Neural theorem provers must both propose tactics and decide which valid successor states to explore. We study whether one-step Lean transitions provide a self-supervised signal for branch ordering. A JEPA-style model predicts latent successor representations and scores only kernel-validated, nonterminal successors generated by a fixed pretrained ByT5 proposer. JEPA achieves higher Top-1 than matched InfoNCE on a same-theorem ranking diagnostic (50.18% versus 31.55%), but averages 282.3 of 987 solved theorems across three seeds versus 308 for proposer ordering, while requiring more tactic checks. In this setting, accurate one-step transition ranking is therefore insufficient as a long-horizon search value. The controlled evaluation separates representation from proposal quality and treats kernel-checked proof completion as the primary endpoint.

## 1 Introduction

Lean proof search alternates between proposing a tactic and asking a trusted kernel to validate its effect on the current state (de Moura et al., 2015). Although modern systems learn the proposal distribution, success also depends on which valid branches receive the remaining search budget (Yang et al., 2023; George et al., 2025; Lamont et al., 2025). We ask whether a one-step proof transition can provide a self-supervised branch-ordering signal without proof-length or success labels.

Proof artifacts have supported self-supervised auxiliary tasks for tactic generation (Han et al., 2022). We instead adapt joint-embedding predictive architectures (JEPAs), which predict target representations rather than target tokens (Assran et al., 2023), to transitions with context $(s, a)$ and recorded successor $s'$. We use an existing pretrained ByT5 tactic generator (Xue et al., 2022; Yang et al., 2023), keep its parameters and deterministic decoding fixed, and use its candidates in every condition; the transition model does not generate tactics. Because proof states and tactics form a structured mathematical language, evaluating representations learned from them is relevant to automated theorem proving, autoformalization, and language-based mathematical reasoning whose outputs require kernel verification.

Our contributions are: (i) a self-supervised latent proof-transition objective with an exponential-moving-average (EMA) target encoder; (ii) a real-kernel beam-search integration in which learned scores never see an unvalidated successor; and (iii) a leakage-controlled comparison with proposer score, hand-designed progress, and a matched InfoNCE objective. Both diagnostic and end-to-end outcomes are reported under the same fixed experimental protocol.

The evaluation separates three claims: distinguishing a recorded successor from states of other theorems, retaining that signal against harder same-theorem alternatives, and improving allocation of a finite kernel-search budget. The first two act as representation diagnostics, only the third measures actual performance.

## 2 Method

### 2.1 Self-Supervised Transition Objective

For a recorded one-step transition $(s, a, s')$, a context encoder $f_\theta$, predictor $q_\phi$, and target encoder $f_\xi$ produce

$$\hat{z} = q_\phi(f_\theta([s; a])), \qquad z' = \mathrm{sg}(f_\xi(s')), \tag{1}$$

where sg stops gradients and $\xi \leftarrow m\xi + (1-m)\theta$. Our JEPA loss is

$$\mathcal{L}_{\mathrm{J}} = 1 - \cos(\hat{z}, z') + \lambda_v \mathcal{L}_{\mathrm{var}}(\hat{z}) + \lambda_c \mathcal{L}_{\mathrm{NCE}}(\hat{z}, z'). \tag{2}$$

We use JEPA for this predictive objective with a low-weight contrastive auxiliary and InfoNCE for the matched direct-contrastive baseline.

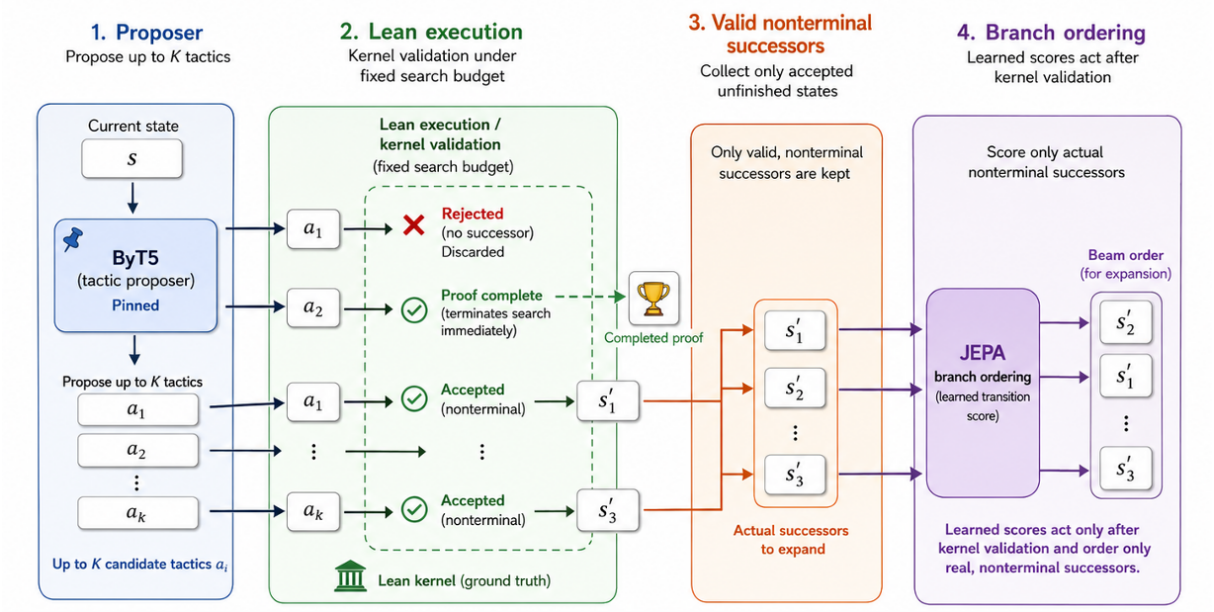


Figure 1: The search pipeline: a pinned ByT5 model proposes up to $K$ tactics, which are executed and validated by Lean; invalid tactics are discarded and completed proofs end the search. The JEPA model scores only valid nonterminal successor states, influencing branch expansion order without affecting tactic generation or correctness.

Following the variance regularizer proposed by Bardes et al. (2022), the variance floor discourages collapse. InfoNCE uses $\mathcal{L}_{\text{NCE}}$ directly; encoders, predictor, EMA, data, optimizer, batch, epochs, and seeds are identical.

### 2.2 Kernel-Gated Beam Search

At each node, ByT5 proposes up to $K$ tactics; Lean rejects invalid tactics before scoring and terminates immediately on a completed proof. Each valid nonterminal successor receives a step score—the proposer's sequence score, $-g(s') - 10^{-6}|s'|$ for progress ($g$ counts goals), or predicted–target cosine—and its beam priority is the sum of step scores along its partial proof. Because scores are not cross-calibrated, comparisons concern each method's induced ordering. We keep the higher-priority path for duplicate printed states, sort descending, retain the top eight, and preserve proposal traversal order on exact ties.

Figure 1 illustrates that JEPA scores only kernel-validated, nonterminal successors and therefore cannot reduce the kernel cost already spent on candidate execution; it only allocates the remaining budget among valid, unfinished branches.

Unlike LeanProgress, which uses supervised remaining-step targets (George et al., 2025), our objective requires only recorded one-step transitions. 3D-Prover predicts multiple tactic properties and performs diversity-aware tactic selection (Lamont et al., 2025), whereas our model orders kernel-produced successor states. Effect-grounded tactic embeddings are evaluated through retrieval or analogy (Samoylov and Vosoughi, 2025); our primary endpoint is completed, kernel-checked proofs.

## 3 Experimental Setup

**Data integrity.** LeanDojo Benchmark 4 contains 259,580 transitions in the official random split (Yang et al., 2023): 250,814 training (96.62%), 4,260 validation (1.64%), and 4,506 test (1.74%). They span 59,557 training, 995 validation, and 992 test theorems, with zero overlap. We use the complete training and test partitions. Imported supervised fine-tuning (SFT) rows match every state pair and identity in the official traced archive. They are treated as independent transitions: only 67.3% of adjacent within-theorem pairs are state-continuous because nested goals do not form a linear trace. The byte-BPE tokenizer is fit on train text only. Empty transition fields and cross-split overlap are zero; the artifact file and every checkpoint are checksummed. Either the state–tactic input or successor exceeds 1,024 tokens in 0.38%, 0.35%, and 0.31% of training, validation, and test rows, respectively.

**Models and training.** The JEPA-style model and direct InfoNCE control both use six Transformer layers, width 512, a 384-dimensional projection, and a three-layer predictor: 22.7M trainable param-

eters plus a 21.5M-parameter EMA target. Maximum length is 1,024 tokens. We train ten epochs at physical batch 32 and gradient accumulation 16 (effective batch 512) for seeds 7, 17, and 37. JEPA uses $m = 0.996$, $\lambda_v = 0.1$, and $\lambda_c = 0.1$. Architecture, optimization, EMA, data, checkpoint selection, and seeds are matched between the two scorers; only the objective differs. InfoNCE uses the other 31 physical-batch examples as negatives; accumulation changes the optimizer batch, not that negative pool. The untrained control instantiates the same architecture and train-fitted tokenizer independently at each seed, without copying learned weights. The supervised control is the fixed published tactic generator's sequence likelihood. We do not fabricate a matched remaining-step target: the imports lack verified distance labels, so deterministic goal-count/state-length progress is the separate value heuristic.

Proposer score preserves the tactic generator's learned prior; progress provides a hand-designed value heuristic; the untrained model isolates effects of the architecture and tokenizer without representation learning; and direct InfoNCE tests whether predictive learning helps beyond contrastive discrimination.

**Ranking evaluation.** Cross-theorem ranking uses all 4,506 test transitions with 16, 32, and 64 negatives from other theorems. Same-theorem ranking uses 1,290 transitions having at least 16 distinct alternatives and evaluates 1, 4, and 16 negatives. A fixed seed gives every checkpoint identical pools within each scope. We report Top-1, MRR, true/negative cosine, and margin.

**Search configuration.** Search uses the existing pretrained LeanDojo ByT5-small tactic generator at pinned revision `67a2c53` (Yang et al., 2023), with frozen parameters and deterministic decoding, LeanDojo 2.1.3, and Lean 4.10.0-rc1. Conditions share theorem keys and the deterministic proposer configuration; learned conditions are run for all three seeds. Completed terms containing `sorry` or unresolved metavariables are rejected before a final kernel type check. The frozen budget is $K = 64$, 512 tactic checks, beam 8, depth 16, and 120 seconds per theorem. Each condition covers all 992 test keys exactly once, with the theorem set partitioned into five deterministic stride shards, each fixed to one RTX 3090 device across two hosts for every condition. Root initialization retains LeanDojo's 600-second default; tactics retain a 60-second limit.

| | Same theorem (16) | | Cross theorem (64) | |
|---|---|---|---|---|
| Objective | Top-1 | MRR | Top-1 | MRR |
| Untrained | 0.0708 | 0.1985 | 0.0200 | 0.0825 |
| JEPA | **0.5018** | **0.6423** | 0.7837 | 0.8057 |
| InfoNCE | 0.3155 | 0.4754 | **0.8105** | **0.8422** |

Table 1: Test transition ranking, reported as three-seed means. Bold marks the best objective within each negative scope; full uncertainties and cosine margins appear in Appendix C.

**Failure handling.** A tactic-level REPL exit consumes its check and triggers at most 10 exact-state recoveries. Exhausting the restart bound invalidates the entire condition. Deterministic zero-check backend-initialization failures are listed and excluded uniformly rather than counted as failed proofs; we report both 992 attempted keys and the eligible denominator. We report theorem-level wins/losses and a paired bootstrap 95% interval for solve-rate differences. Any missing key or budget mismatch invalidates a condition. An unrecoverable in-search kernel-process error terminates that theorem as an unsolved failure and is reported per condition rather than excluded. We predefine material check efficiency as identical solved count in every seed, at least 10% fewer ordinary checks in every seed, a theorem-clustered check interval below zero, and no increase in theorem timeouts. We also report valid/checked tactic applications (`step_success`) and termination/failure counts.

**Statistical analysis.** The theorem, rather than a tactic application or training seed, is the unit of paired inference. For learned conditions we first average the three outcomes within each theorem and then resample theorem keys, retaining their matched outcomes across search orders. This avoids treating correlated seeds or the many checks inside one proof attempt as independent evidence. The primary claim is fixed before inspecting test outcomes. A positive result requires more solved theorems, or exactly matched solves with the preregistered check reduction, however ranking metrics alone cannot satisfy it.

## 4 Results and Discussion

Table 1 reports the preregistered representation diagnostics. On the harder same-theorem population of 1,290 transitions, JEPA achieves $0.5018 \pm 0.0072$ Top-1, compared with

| Search order | Solved | Checks | Wall (s) |
|---|---|---|---|
| Proposer score | **308/987** | **349.92** | **64.62** |
| Progress | 304/987 | 352.99 | 65.31 |
| Untrained | 303.7±8.5/987 | 351.03±2.86 | 65.99±0.30 |
| InfoNCE | 289.7±3.1/987 | 359.89±0.50 | 66.02±0.03 |
| JEPA | 282.3±1.5/987 | 363.45±1.02 | 66.61±0.08 |

Table 2: Fixed-budget Lean-kernel search. Checkpoint rows report three-seed means and sample standard deviations. Bold marks the best value: higher for solved theorems and lower for checks and wall time. Paired comparisons use theorem-level outcomes.

$0.3155 \pm 0.0074$ for InfoNCE and $0.0708 \pm 0.0256$ for untrained weights, and leads both controls in every seed. Holding theorem identity constant removes an easy retrieval shortcut, although the task remains action-conditioned and does not measure proof completion.

With 64 cross-theorem negatives, InfoNCE instead achieves the highest Top-1 ($0.8105 \pm 0.0072$), followed by JEPA ($0.7837 \pm 0.0020$). The reversal shows that objective rankings depend on negative construction; neither diagnostic alone establishes a useful long-horizon search ordering.

Table 2 gives the primary endpoint. JEPA solves $282.3 \pm 1.5$ of 987 eligible theorems, compared with 308 for proposer ordering. The theorem-clustered mean difference is $-25.67$ solves ($-2.600$ percentage points; 95% interval $[-3.715, -1.554]$), accompanied by 13.53 additional checks per theorem (95% interval $[9.48, 17.85]$). JEPA also trails progress, untrained weights, and InfoNCE; complete paired comparisons appear in Appendix 9. One in-search backend error in each of InfoNCE seed 7 and JEPA seed 17 is conservatively counted as unsolved, with no condition-specific exclusion.

The preregistered positive gate is not satisfied. Under this proposer and budget, accurate ranking of observed local transitions does not produce a useful ordering for proof completion. This result is consistent with an objective mismatch: transition consistency characterizes the immediate effect of a tactic, whereas search requires the long-horizon value of the resulting branch.

The matched untrained model stays close to proposer and progress, while both trained representations solve fewer theorems. Because untrained, JEPA, and InfoNCE share scoring cost and architecture, inference overhead alone is insufficient to explain the learned models' larger solve deficits. The data do not identify a unique mechanism, but they localize the failure to the ordering induced by the learned objectives under this candidate distribution. In particular, a locally predictable successor may preserve the surface form and tactic effect of its parent without being a strategically valuable state from which to finish the theorem.

These findings indicate that held-out transition retrieval should be treated as a representation diagnostic rather than a prover result. Kernel-aligned evaluation must additionally fix the candidate source, charge every tactic execution to a common budget, and report completed proofs and checks. Longer-horizon or set-valued JEPA targets remain hypotheses requiring separate preregistered evaluation.

## 5 Conclusion

We evaluate JEPA-style proof-transition learning as an ordering mechanism for successors produced by Lean. Fixing the generator, theorem set, and kernel-check budget separates representation quality from proposal quality and replay artifacts. Across three seeds, JEPA provides the strongest same-theorem transition ranking but solves fewer proofs and consumes more checks than every control. The study therefore yields a controlled negative result under this configuration for one-step latent prediction as a branch-ordering score, without addressing longer-horizon JEPA objectives.

## Limitations

Our results are limited to one small encoder, one tactic generator, and one random split. The SFT rows contain independent one-step transitions rather than verified trajectories, precluding trajectory objectives and a learned remaining-step control. Successor scoring follows kernel execution, long states are truncated at the reported rates, and wall-clock measurements are hardware-dependent; the tactic-check budget and paired solved set are therefore the primary controls. Training and search also differ in transition distribution: recorded one-step SFT examples supervise the model, whereas search states are induced by the fixed ByT5 candidate set. The design does not distinguish objective mismatch from out-of-distribution scoring. In addition, each ordering accumulates its native step score, so depth preferences can differ even though the aggregation rule is fixed before test evaluation. Random-split library similarity may also limit generalization; all require separate evaluation.

## References


Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. 2023. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15619–15629.

Adrien Bardes, Jean Ponce, and Yann LeCun. 2022. VICReg: Variance-invariance-covariance regularization for self-supervised learning. In *International Conference on Learning Representations*.

Leonardo de Moura, Soonho Kong, Jeremy Avigad, Floris van Doorn, and Jakob von Raumer. 2015. The lean theorem prover (system description). In *Automated Deduction—CADE-25*, pages 378–388. Springer.

Robert Joseph George, Suozhi Huang, Peiyang Song, and Anima Anandkumar. 2025. LeanProgress: Guiding search for neural theorem proving via proof progress prediction. *Transactions on Machine Learning Research*.

Jesse Michael Han, Jason Rute, Yuhuai Wu, Edward W. Ayers, and Stanislas Polu. 2022. Proof artifact co-training for theorem proving with language models. In *International Conference on Learning Representations*.

Sean Lamont, Christian Walder, Amir Dezfouli, Paul Montague, and Michael Norrish. 2025. 3D-Prover: Diversity driven theorem proving with determinantal point processes. In *Advances in Neural Information Processing Systems*, volume 38.

Elisaveta Samoylov and Soroush Vosoughi. 2025. Modeling tactics as operators: Effect-grounded representations for lean theorem proving. In *Proceedings of the 3rd Workshop on Mathematical Natural Language Processing*, pages 168–175. Association for Computational Linguistics.

Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. 2022. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306.

Kaiyu Yang, Aidan Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan J. Prenger, and Animashree Anandkumar. 2023. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems*, volume 36.

## A Evidence Chain

The study follows one question: does learning a valid one-step transition signal improve the order in which a prover spends its search budget? The appendix follows that question from representation learning to the kernel-checked endpoint.

## B Model and Evaluation Boundary

The target encoder receives no optimizer gradient. During search, JEPA scores only nonterminal successors returned by Lean; invalid tactics are already discarded and completed proofs already terminate. Thus scoring can reallocate the remaining budget, but cannot recover the check spent producing a successor.

The best validation-loss checkpoint is selected within each fixed ten-epoch run. Accumulation changes the optimizer batch, not the 31-example in-batch negative pool. Random controls are independently initialized at each seed.

## C Local Transition Evidence

The first large JEPA collapsed: true and negative successor cosines were both $0.9537$, yielding essentially zero margin. A variance floor and low-weight contrastive auxiliary restored separation. The early rows below explain the model-development path; only the official three-seed rows support claims.

The official audit then fixed the unit of evidence. All 259,580 imported state pairs and identities match the traced archive, theorem overlap is zero, and the tokenizer is trained only on the training split. Encounter order is not a proof trajectory: only 67.27% of adjacent within-theorem pairs are state-continuous, often because nested goals reappear after a printed `no goals`. Training therefore uses independent one-step transitions, while inference uses states returned directly by Lean.

Cross-theorem negatives favor InfoNCE, whereas same-theorem negatives favor JEPA in every seed. Holding theorem identity fixed removes one retrieval shortcut, but this remains an action-conditioned local diagnostic rather than proof completion.

### C.1 Post-Hoc Auxiliary-Loss Ablation

Removing the auxiliary term isolates latent prediction with the same data, architecture, optimizer, EMA, training budget, and three seeds.

The drop and larger cross-seed variation show that the low-weight contrastive term materially stabilizes the reported transition signal. No completed kernel-search condition exists for $\lambda_c = 0$, so this table supports only a representation-level conclusion.

## D Kernel-Checked Search

The operational test fixes Lean 4.10.0-rc1, LeanDojo 2.1.3, ByT5-small revision `67a2c53`, 64 candidates, 512 ordinary checks, beam 8, depth 16, and a 120-second theorem limit. Completed terms containing `sorry`, unresolved metavariables, or a failed final declaration check are rejected. REPL recovery replays the exact path, verifies state equality, and never enlarges the budget.

Five deterministic zero-check initialization failures are excluded uniformly, leaving 987 eligible theorems from 992 attempted keys. In-search backend errors remain eligible unsolved outcomes.

## E Claim Scope and Reproducibility

The evidence chain supports a narrow conclusion: one-step latent prediction learns theorem-local transition structure, but that structure is not a useful long-horizon branch value under this proposer and budget. It does not rule out longer-horizon, set-valued, or search-trained JEPA targets.

The complete test suite covers target stop-gradient and EMA-only updates, official theorem disjointness, exact transition provenance, real Lean-successor scoring, unsafe-proof rejection, restart accounting, shard completeness, clustered uncertainty, and manuscript-result linkage. Frozen artifact and protocol hashes preserve the full audit trail.

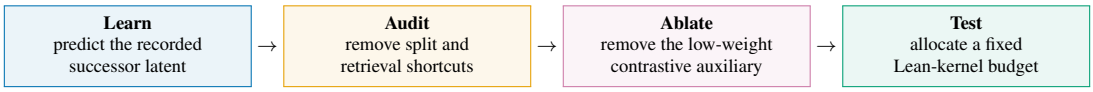


Figure 2: The evidence chain. Local ranking establishes representation quality; only completed, kernel-checked proofs establish search value.

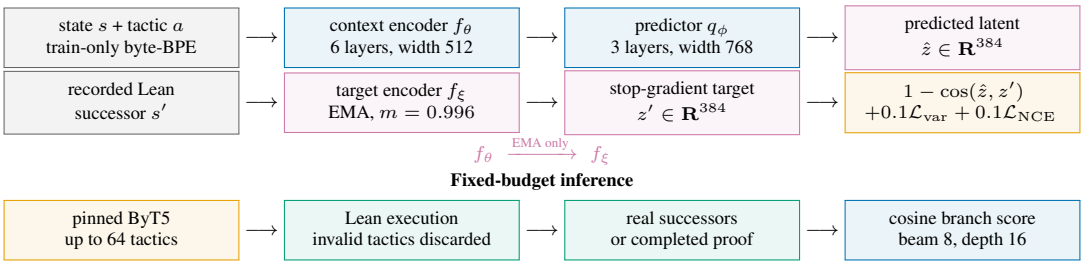


Figure 3: Training and inference boundary. The learned model predicts and orders representations; Lean alone creates successors and certifies proof completion.

| Setting | Value | Setting | Value |
|---|---|---|---|
| Tokenizer | byte-BPE, train only | Maximum length | 1,024 |
| Vocabulary / min. frequency | 16,000 / 2 | Encoder layers / heads | 6 / 8 |
| Hidden / feed-forward width | 512 / 1,024 | Projection / predictor width | 384 / 768 |
| Predictor layers / dropout | 3 / 0.15 | Trainable / EMA parameters | 22.7M / 21.5M |
| Epochs / seeds | 10 / 7, 17, 37 | Physical / effective batch | 32 / 512 |
| Learning rate / weight decay | $10^{-5}$ / $10^{-4}$ | Gradient clip / accumulation | 1.0 / 16 |
| EMA / InfoNCE temperature | 0.996 / 0.05 | Variance / auxiliary weight | 0.1 / 0.1 |

Table 3: Matched model and optimization settings. Direct InfoNCE changes the objective; untrained controls use the same architecture and tokenizer.

| Stage | Objective / split | Negatives | Top-1 | MRR | Margin |
|---|---|---|---|---|---|
| Initial large model | cosine, artifact split | 16 | 0.0550 | 0.2027 | −0.0000 |
| Stabilized model | cosine + var. + NCE, artifact split | 16 | 0.8944 | 0.9168 | 0.3054 |
| Clean JEPA | same loss, theorem split | 16 | 0.8862 | 0.9092 | 0.3620 |
| Clean InfoNCE | direct contrastive, theorem split | 16 | 0.9220 | 0.9548 | 0.2874 |

Table 4: Development diagnostics before the official full-data matrix. Populations differ, so rows are provenance rather than a controlled comparison.

| Split | Transitions | Theorem keys | Either side over 1,024 tokens |
|---|---|---|---|
| Train | 250,814 | 59,557 | 0.38% |
| Validation | 4,260 | 995 | 0.35% |
| Test | 4,506 | 992 | 0.31% |

Table 5: Official Benchmark 4 transition split. Cross-split theorem overlap, empty fields, and source-pair mismatches are all zero.

| Negative scope | Objective | Top-1 | MRR | True cosine | Neg. cosine | Margin |
|---|---|---|---|---|---|---|
| Cross-theorem, 64 | Untrained | 0.0200±0.0045 | 0.0825±0.0075 | −0.0222±0.0142 | −0.0236±0.0126 | 0.0013±0.0021 |
| Cross-theorem, 64 | JEPA | 0.7837±0.0020 | 0.8057±0.0024 | 0.9350±0.0095 | 0.6318±0.0492 | 0.3032±0.0398 |
| Cross-theorem, 64 | InfoNCE | **0.8105±0.0072** | **0.8422±0.0056** | 0.4281±0.0151 | 0.0870±0.0211 | **0.3411±0.0062** |
| Same-theorem, 16 | Untrained | 0.0708±0.0256 | 0.1985±0.0251 | −0.0205±0.0138 | −0.0095±0.0139 | −0.0110±0.0056 |
| Same-theorem, 16 | JEPA | **0.5018±0.0072** | **0.6423±0.0063** | 0.9403±0.0101 | 0.8178±0.0408 | **0.1225±0.0307** |
| Same-theorem, 16 | InfoNCE | 0.3155±0.0074 | 0.4754±0.0090 | 0.4730±0.0157 | 0.4282±0.0188 | 0.0448±0.0032 |

Table 6: Hardest official ranking scopes, reported as mean and sample standard deviation over seeds. Raw cosine levels are objective-dependent; rank and within-objective margins are the relevant diagnostics.

| Objective | Cross Top-1@64 | Cross MRR@64 | Same Top-1@16 | Same MRR@16 |
|---|---|---|---|---|
| JEPA, $\lambda_c = 0.1$ | $\mathbf{0.7837 \pm 0.0020}$ | $\mathbf{0.8057 \pm 0.0024}$ | $\mathbf{0.5018 \pm 0.0072}$ | $\mathbf{0.6423 \pm 0.0063}$ |
| JEPA, $\lambda_c = 0$ | $0.3880 \pm 0.1347$ | $0.4810 \pm 0.1289$ | $0.3721 \pm 0.0237$ | $0.5113 \pm 0.0269$ |

Table 7: Requested loss ablation on the predefined representation diagnostics. Bold marks the stronger mean within each column.

| Search order | Solved | Step success | Checks/thm. | Wall s/thm. | Timeouts | Errors |
|---|---|---|---|---|---|---|
| Proposer score | 308/987 | 26.59% | 349.92 | **64.62** | 83 | **0** |
| Progress | 304/987 | **28.26%** | 352.99 | 65.31 | 89 | **0** |
| Untrained seed 7 | 294/987 | 25.90% | 353.52 | 66.23 | 90 | **0** |
| Untrained seed 17 | 307/987 | 26.40% | 351.65 | 66.08 | 91 | **0** |
| Untrained seed 37 | **310/987** | 26.16% | **347.91** | 65.65 | 93 | **0** |
| JEPA seed 7 | 284/987 | 25.16% | 364.61 | 66.52 | 84 | **0** |
| JEPA seed 17 | 282/987 | 25.42% | 362.68 | 66.67 | 89 | 1 |
| JEPA seed 37 | 281/987 | 24.66% | 363.05 | 66.65 | 85 | **0** |
| InfoNCE seed 7 | 289/987 | 24.88% | 360.41 | 66.00 | **81** | 1 |
| InfoNCE seed 17 | 287/987 | 25.32% | 359.85 | 66.06 | 85 | **0** |
| InfoNCE seed 37 | 293/987 | 25.00% | 359.41 | 66.02 | 88 | **0** |

Table 8: Complete fixed-protocol outcomes. Higher is better for solved and step success; lower is better elsewhere. Errors are retained as unsolved.

| Comparator | JEPA $\Delta$ solves | $\Delta$ success (points) | 95% interval | Wins/losses/ties |
|---|---|---|---|---|
| Proposer score | $-25.67$ | $-2.600$ | $[-3.715, -1.554]$ | 6/36/945 |
| Progress | $-21.67$ | $-2.195$ | $[-3.445, -1.047]$ | 13/35/939 |
| Untrained | $-21.33$ | $-2.161$ | $[-3.073, -1.317]$ | 12/49/926 |
| InfoNCE | $-7.33$ | $-0.743$ | $[-1.317, -0.236]$ | 6/20/961 |

Table 9: Theorem-clustered paired comparisons. Negative values favor the comparator; learned conditions are averaged within theorem over seeds.

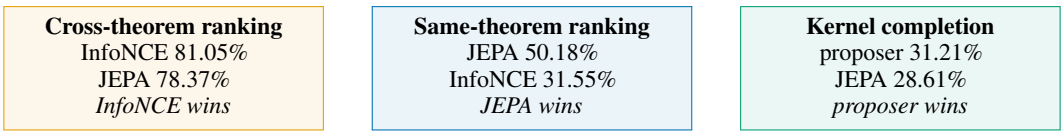


Figure 4: The proxy reversal. Either representation objective can win a local diagnostic while neither induces the best proof-search order.

| Gate | Failure condition | Accepted evidence |
|---|---|---|
| Data | split overlap, empty state, source mismatch | 259,580/259,580 exact pairs |
| Ranking | incomplete matrix or negative-pool drift | nine linked checkpoints |
| Kernel | missing key, budget drift, malformed error | 11 conditions, 992 keys each |
| Pairing | unmatched theorem set or seed as sample | theorem-clustered comparison |
| Proof validity | `sorry`, metavariable, declaration failure | kernel-checked completion |
| Manuscript | placeholder, identifier, unembedded font, cell drift | exact result handoff |

Table 10: Fail-closed gates connecting imported data, checkpoints, search outcomes, and reported claims.